

实验3 类的运算符重载和输入输出

一. 实验目的

1. 掌握运算符重载方法
2. 掌握运算符重载规则
3. 掌握不同数据类型间的转换
4. 能利用运算符重载知识，解决实际问题。

二. 实验内容

1. 设计 RationalNumber (分数类)

- 1) 建立构造函数，它能防止分母为 0，当分数不是最简形时进行约分以及避免分母为负数
- 2) 重载加法、减法、乘法以及除法运算符
- 3) 重载关系运算符(<, >)和相等运算符
- 4) 能利用类型转换方法，实现整数类型与分数相加

完成上述分数类，使得下面的主函数能够运行并得到图中的测试结果。

```
int main()
{
    cout<<"请输入两个分数 a,b(分子和分母): "<<endl;
    int x,y,m,n;
    cout<<"a:"<<cin>>x>>y;RationalNumber a(x,y);
    cout<<"b:"<<cin>>m>>n;RationalNumber b(m,n);
    RationalNumber c(1,1);
    int k;
    cout<<"请输入一个整数: "<<endl;
    cin>>k;
    cout<<"整数 d:"<<endl;
    cout<<k<<endl;
    cout<<"两个分数相加: " <<endl;
    c=a+b;c.display();
    cout<<endl;
    cout<<"两个分数相减: " <<endl;
    c=a-b;c.display();
    cout<<endl;
    cout<<"两个分数相乘: " <<endl;
    c=a*b;c.display();
    cout<<endl;
    cout<<"两个分数相除: " <<endl;
    c=a/b;c.display();
    cout<<endl;
    cout<<"两个分数比较: " <<endl;
    compare(a,b);
    cout<<"分数 a 和整数 d 相加: " <<endl;
    RationalNumber c3(k);
    c=a+c3;c.display();
    return 0;
}
```

}

请输入两个分数a,b(分子和分母):

a:1 2

b:2 3

请输入一个整数:

2

整数d:

2

两个分数相加:

7/6

两个分数相减:

-1/6

两个分数相乘:

1/3

两个分数相除:

3/4

两个分数比较:

1/2<2/3

分数a和整数d相加:

5/2

2. 设计一个银行账户类 `BankAccount`，模拟账户的基本操作。包含以下成员：*
- 私有数据成员：账号 `string aID`; 余额（元） `double balan`; 年利率（如 0.03 表示 3%） `double aRate`;
 - 构造函数：带默认参数的构造函数，用于初始化账号、余额和年利率（年利率默认为 0.03）
 - 运算符重载：
 - 友元函数 `operator>>`：重载流提取运算符，实现账户信息的输入（账号、余额、年利率）
 - 友元函数 `operator<<`：重载流插入运算符，实现账户信息的输出（格式见示例）
 - 前置自增运算符 `++`：将账户余额增加一个月的利息（余额 *= (1 + 年利率/12) ），返回当前对象的引用
 - 后置自增运算符 `++`：同样增加一个月利息，但返回自增前的对象副本
 - 类型转换 `operator double()`：返回当前余额，允许对象直接当作 `double` 使用
 - 加法运算符 `+`：实现存款，支持 对象+ 金额 和 金额 + 对象（注意：一个成员函数，一个友元函数）

完成该 `BankAccount` 类，使得下面的主函数能够运行并得到图中的测试结果。注意：账户余额的输出格式保留两位小数。

```
int main() {
    cout << "===== 测试 1: 输入输出运算符 =====" << endl;
    BankAccount myAcc;
    cin >> myAcc;
    cout << "\n 您创建的账户信息如下: " << endl;
    cout << myAcc << endl;
    cout << "\n===== 测试 2: 前置自增 ++ (增加一个月利息) =====" << endl;
    cout << "前置++前余额: " << (double)myAcc << " 元" << endl;
    ++myAcc;
    cout << "前置++后余额: " << myAcc << endl;
    cout << "\n===== 测试 3: 后置自增 ++ =====" << endl;
    BankAccount oldAcc = myAcc++;
    cout << "后置++后 myAcc 余额: " << myAcc << endl;
    cout << "后置++返回的旧账户: " << oldAcc << endl;
    cout << "\n===== 测试 4: 类型转换 (BankAccount -> double) =====" << endl;
```

```

double bal = myAcc;
cout << "直接转换为 double 的余额:" << bal << " 元" << endl;
cout << "\n===== 测试 5: 存款 (加法运算符) =====" << endl;
BankAccount newAcc = myAcc + 500.0;
cout << "原账户:" << myAcc << endl;
cout << "存款 500 元后的新账户:" << newAcc << endl;
BankAccount anotherAcc = 200.0 + myAcc;
cout << "使用 200.0 + 账户 的结果:" << anotherAcc << endl;

return 0;
}
===== 测试1: 输入输出运算符 =====
请输入账号: test
请输入余额: 1000
请输入年利率 (如 0.03 表示 3%): 0.03

您创建的账户信息如下:
账号: test, 余额: 1000.00 元, 年利率: 3.00%

===== 测试2: 前置自增 ++ (增加一个月利息) =====
前置++前余额: 1000.00 元
前置++后余额: 账号: test, 余额: 1002.50 元, 年利率: 3.00%

===== 测试3: 后置自增 ++ =====
后置++后 myAcc 余额: 账号: test, 余额: 1005.01 元, 年利率: 3.00%
后置++返回的旧账户: 账号: test, 余额: 1002.50 元, 年利率: 3.00%

===== 测试4: 类型转换 (BankAccount -> double) =====
直接转换为 double 的余额: 1005.01 元

===== 测试5: 存款 (加法运算符) =====
原账户: 账号: test, 余额: 1005.01 元, 年利率: 3.00%
存款500元后的新账户: 账号: test, 余额: 1505.01 元, 年利率: 3.00%
使用 200.0 + 账户 的结果: 账号: test, 余额: 1205.01 元, 年利率: 3.00%

```

3. 编程求 100 以内的所有素数（质数，除 1 和它本身外无其他因数）。将所得数据存入文本文件和二进制文件。对写入文本文件的素数，要求存放格式是每行 10 个素数，每个数占 6 个字符，左对齐。二进制文件整型数的长度用 `sizeof()` 来获得，要求可以正序读出，也可以逆序读出（利用文件定位指针移动实现），读出数据按文本文件中的格式输出显示。提示：二进制文件读取时，读取完成可添加一行代码 `输入流对象.clear(0);` 这行代码可以将文件输入流状态位恢复初始状态，以便下次读取。

4. 编写一个程序，利用二进制文件读取和写入商品的商品号、商品名、单价和数量。商品的初始信息如下表。

Id	Name	Number	Price
001	田字格本	80	1.2
002	纸巾	20	12.1
003	可乐	15	35.6
004	保温杯	20	22.8
005	农谷山泉	120	2.0

主函数中要求实现以下功能：

- 定义一个结构体 `Product`，包含成员：商品号 `char id[6]`，商品名 `char name[20]`，数量 `int number`，单价 `double price`
- 在程序中用该结构体创建一个包含 5 个元素的数组，并用表格中数据初始化，以二进制

方式写入文件 `f.data` 中

- 添加新商品：提示用户输入两个新商品，以追加方式(`ios::app`)将新商品写入文件末尾。
- 显示所有商品：从文件中顺序读取所有商品，并以表格形式输出到屏幕（要求左对齐，控制列宽，单价保留两位小数）。
- 逆序输出所有商品：从文件末尾向前依次读取每个商品，并输出到屏幕（使用 `seekg` 和 `tellg` 移动文件指针），输出格式同上
- 查找并修改单价：提示用户输入一个商品号，在文件中查找该商品。若找到，则显示其原信息，然后让用户输入新的单价，将修改后的信息写回原位置（使用 `seekp` 和 `write`）；若未找到，输出提示信息。

提示：显示所有商品功能可以分为显示表格头和显示表格内容两部分，比如显示表格头的函数可以定义如下：

```
// 显示表格头
void displayHeader() {
    cout << left << setw(6) << "商品号"
        << setw(20) << "商品名"
        << setw(8) << "数量"
        << setw(10) << "单价(元)" << endl;
    cout << string(44, '-') << endl;
}
```

三. 实验要求

独立完成实验内容，按照实验报告要求提交。实验报告中应有本次实验的体会。